Observer pattern. This is a fancy way of saying that an object maintains a list of sub-objects, which are dependent on the aforesaid master object, and that the sub-objects are automatically notified of any changes in the state, necessary for them to do their job. To achieve that effect, Qt uses a style that must be very familiar to developers of event-based systems—objects emit *signals* (Qt decided to add an *emit* keyword to C++ for this), which are then sent to *slots* that are listening for those signals.

For this, Qt adds three more access modifiers that you can (and must) use to define these additional pieces of functionality into your classes: *signals*, under which you define the signals that your object can emit, and *public slots* and *private slots*, under which you define the functions implemented by your object to handle signals that your object is listening for.

You don't have to have both signals and slots in all your objects; indeed, you might have an object that only emits certain signals, or you might have objects that only listen for signals emitted by other objects. You might also have objects that do neither, but you'll need to know about how signals and slots work because Qt's bundled libraries use this mechanism extensively.

Both signals and slots are fully inheritable, and can be overloaded just like normal C++ functions. However, there are two factors to consider—one, neither signals nor slots can have a non-void return type, and two, while you have to both declare and define slots, you only need to declare the signal prototype in the class definition, not define it anywhere. In fact, you must not write the function body for the signals anywhere. That work is done somewhere else, as I will explain.

Part 2: QObject and the meta-object system

So how does all of this magic work? Let me explain with some code:

```
class TCPServer : public QObject
{
    Q_OBJECT

    private:

    QTcpServer * server;

    public:

    explicit TCPServer(QObject *parent = 0);
    ~TCPServer();

    signals:

    void haveRequest(QTcpSocket *);
    void haveResponse(QTcpSocket *);

    public slots:

    void newConnection();
    void acceptError()
};
```

The above code snippet is a typical 'Qt-fied' class. What's the first thing you notice about it?

The very first thing that a class must do, if it has to use all of the goodies provided by Qt, is to inherit from the *QObject* class. This sets up a bunch of internal tables and mechanisms that set up the class so that it's usable with the signals and slots system.

The second thing that you must do, before you declare any other variables or methods inside the class, is to write the macro *Q_OBJECT*. Among other things, this tells the meta-object compiler to enable introspection on this class.

C++ is a compiled, static language. It doesn't offer all the goodies of other more user-friendly languages like Python, where you can type *dir(my_object)* and see exactly what's inside the object, or use methods like *hasattr(my_object, "my_method")*, which returns true if *my_object* has a member function called *my_method*. In other words, with C++, you can't inspect the contents of the class during runtime and make decisions based on what you find (this is called introspection, by the way).

C++ doesn't do introspection (this is not strictly true, since there is a mechanism called RTTI - Run Time Type Information - which provides a limited capability to infer type information about objects during runtime, but it requires special compiler support and… let's just say people don't like to use this feature). But the people who built Qt decided that if C++ did have a way of introspection for an object, the signals and slots mechanism could be implemented very elegantly. Indeed, during runtime, when objects keep getting created, and slots rapidly subscribe to and unsubscribe from signals, for the mechanism that marshals all these connections, introspection comes in useful.

So Qt's designers decided to outfit C++ with an introspection mechanism, and the way they did it was pretty cool.

Just before the file is compiled, it's passed through the Meta-Object Compiler or MOC. The MOC is like a pre-processor on steroids. It's a full-fledged C++ code generator, and one of the things it does is take a look at the class contents and build a table inside the class with information on what's in the class. That way, the signals and slots marshalling mechanism needs only to refer to this table to see the internal details of the class, and we have introspection.

The MOC does a lot of other things too. It sets up tables that are used to keep track of the connections between signals and slots. It sets up code that's used to delay deletion of an object until all signal-slot connections that refer to the object are disconnected. This way, it manages to implement